

Securing Model Endpoints with IAM

Protecting your **models, data, and budgets** through core pillars of **Authentication, Authorization, Least Privilege, and Audit** across AWS / GCP / Azure / K8s / Mesh



Threats to Model Endpoints

Key Theft

Unlimited usage, data exfiltration, complete account takeover

Abuse Vectors

DDoS attacks, prompt injection leading to sensitive data leakage

Tenant Breakout

Cross-tenant access in multi-tenant clusters, data contamination

Model Extraction

Query scraping to rebuild proprietary models, extreme cost drain

Configuration Errors

Misconfigured public endpoints exposing sensitive model capabilities

IAM Fundamentals

Authentication (AuthN)

Prove who you are via:

- API keys and secrets
- OpenID Connect (OIDC)
- Mutual TLS (mTLS)

Authorization (AuthZ)

Define what you can do via:

- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC)
- Policy enforcement points

Accountability

- Immutable audit logs
- Distributed trace IDs
- Nonrepudiation through signing

Least Privilege + JIT

- Minimal permissions necessary
- Just-in-time credential issuance
- Time-bound access windows



Recommended Control Layers

Defense-in-Depth Strategy

Edge

WAF, rate limits, bot detection, DDoS protection

API

OAuth2/OIDC, JWT validation, scoped API keys, mTLS

Service Mesh

mTLS, SPIFFE identities, policy enforcement

Network

Private endpoints, VPC peering, egress control

Data

KMS/HSM keys, row/column-level access controls

Audit

Immutable logs, anomaly detection, alert pipelines

Identity Patterns for Model APIs



User-to-API

OAuth2/OIDC flows producing short-lived JWT access tokens



Service-to-Service

mTLS + workload identity using SPIFFE/SPIRE



Machine-to-Cloud

OIDC federation mapped to cloud IAM roles



Break-Glass

Time-bound, heavily audited emergency tokens



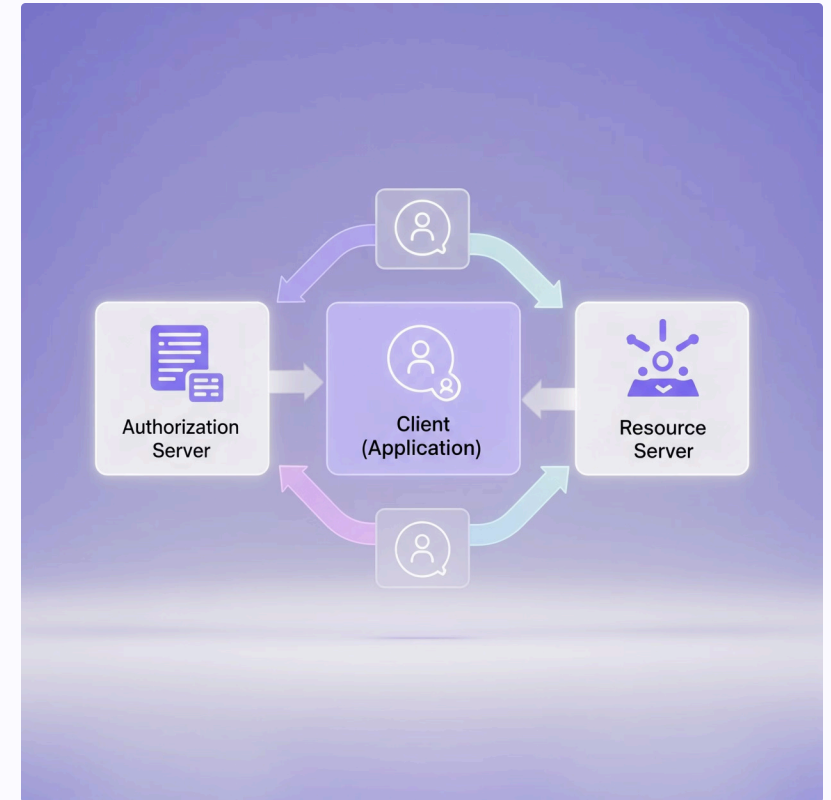
OAuth2/OIDC (Public Clients)

Key Components

- Authorization Code with PKCE (mobile/web)
- JWT contents: `sub`, `aud`, `exp`, `scope`, `tenant_id`
- **Scopes** examples: `model:infer`, `model:admin`, `tokens:read`
- Rotate signing keys (JWKS); clock skew $\leq 2-5$ min

JWT Verification (Python)

```
from jose import jwt
claims = jwt.decode(token, jwks,
    audience="inference-api",
    options={"verify_aud": True})
assert "model:infer" in claims.get("scope", "").split()
```



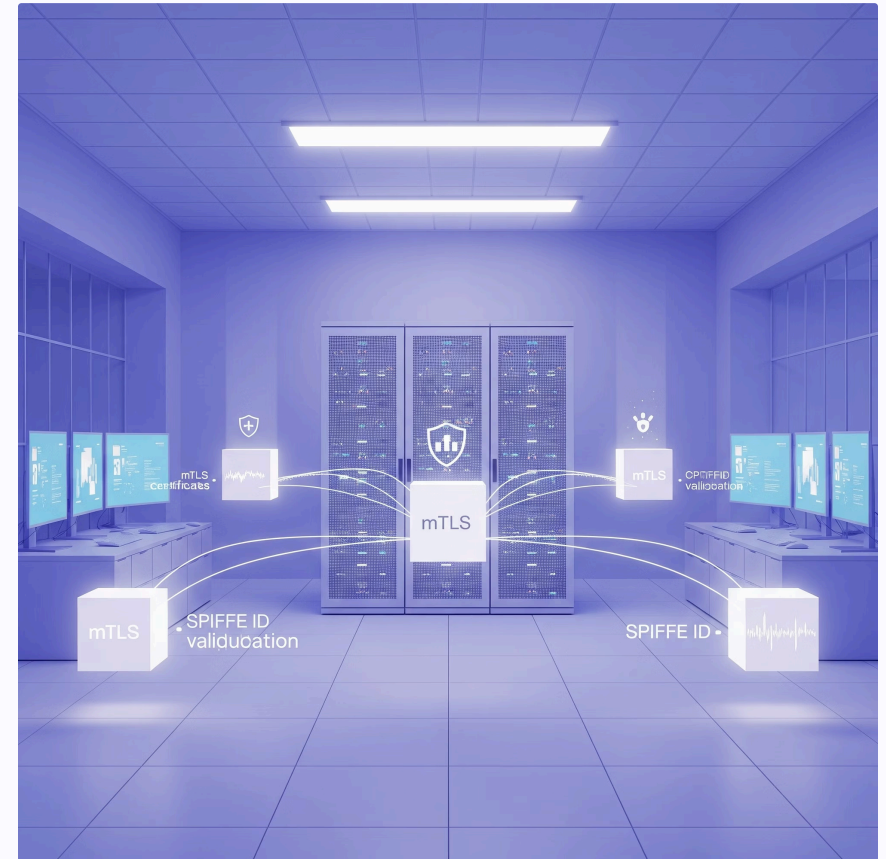
mTLS + SPIFFE (Service-to-Service)

Implementation

- Issue **SPIFFE IDs** (e.g., `spiffe://prod/ns/ml/sa/infer`)
- Service mesh (Istio/Linkerd) handles cert rotation & TLS
- AuthZ via mesh policies: allow only specific SPIFFE IDs

Istio AuthorizationPolicy

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-front-to-infer
  namespace: ml
spec:
  selector:
    matchLabels: {app: infer}
  rules:
    - from:
        - source:
            principals: ["spiffe://prod/ns/gw/sa/ingress"]
```



A conceptual diagram of an API Gateway. It features a stack of five translucent, glowing blue layers. The bottom layer is a solid blue base labeled 'API Gateway' in white. Above it are four more layers, each containing a glowing blue square icon with a white geometric pattern. To the left, a blue rectangular box labeled 'FRONTEND' is partially visible. To the right, a blue rectangular box labeled 'BACKEND' is partially visible. White lines represent data flow, entering from the left and passing through the layers towards the right. The entire scene is set against a dark blue background with a subtle grid pattern.

API Gateway Hardening

JWT Validation

Enforce token validation, required scopes, and proper audience claims

Rate Limiting & Quotas

Implement per-API key and per-tenant limits to prevent abuse

WAF Protection

Configure rules against SQLi, XSS, SSRF, and path traversal attacks

Payload Protection

Enforce request size caps and JSON schema validation

OpenAPI Security Configuration

```
components:
  securitySchemes:
    bearerAuth: {type: http, scheme: bearer, bearerFormat: JWT}
  security:
    - bearerAuth: []
```

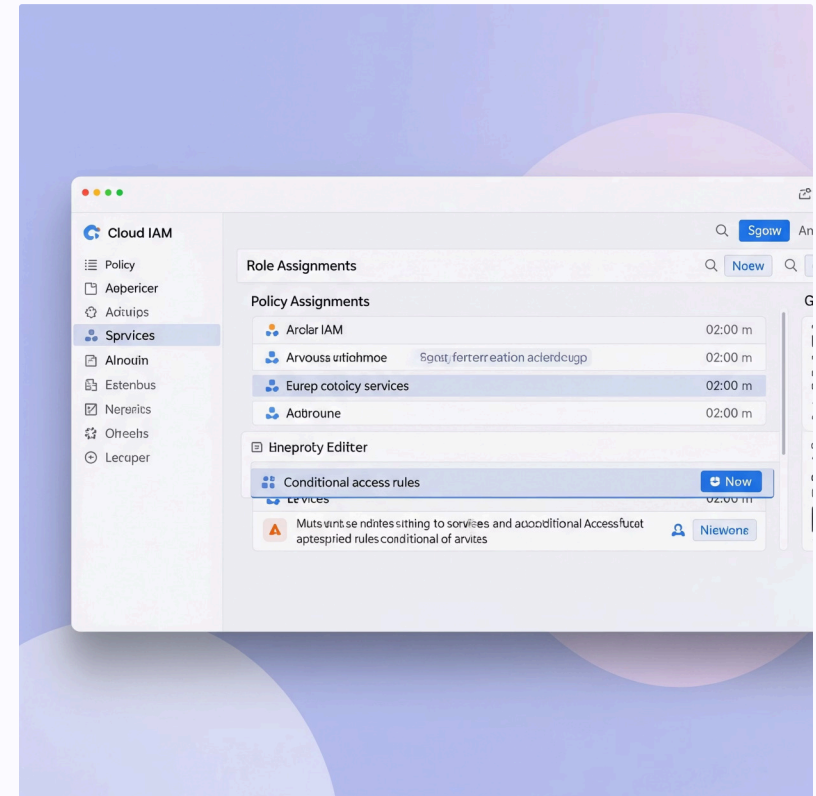
Cloud IAM Quick Patterns

Platform Patterns

- **AWS:** SigV4, IAM roles, STS, API Gateway + Lambda authorizers
- **GCP:** Service Accounts, IAM Conditions, IAP, Workload Identity
- **Azure:** Managed Identities, AAD App Roles, APIM policies
- Prefer **workload identity federation** over static keys

AWS IAM Policy (Least Privilege)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::capstone-models/prod/*",
    "Condition": {
      "StringEquals": {
        "aws:PrincipalTag/tenant": "acme"
      }
    }
  }]
}
```



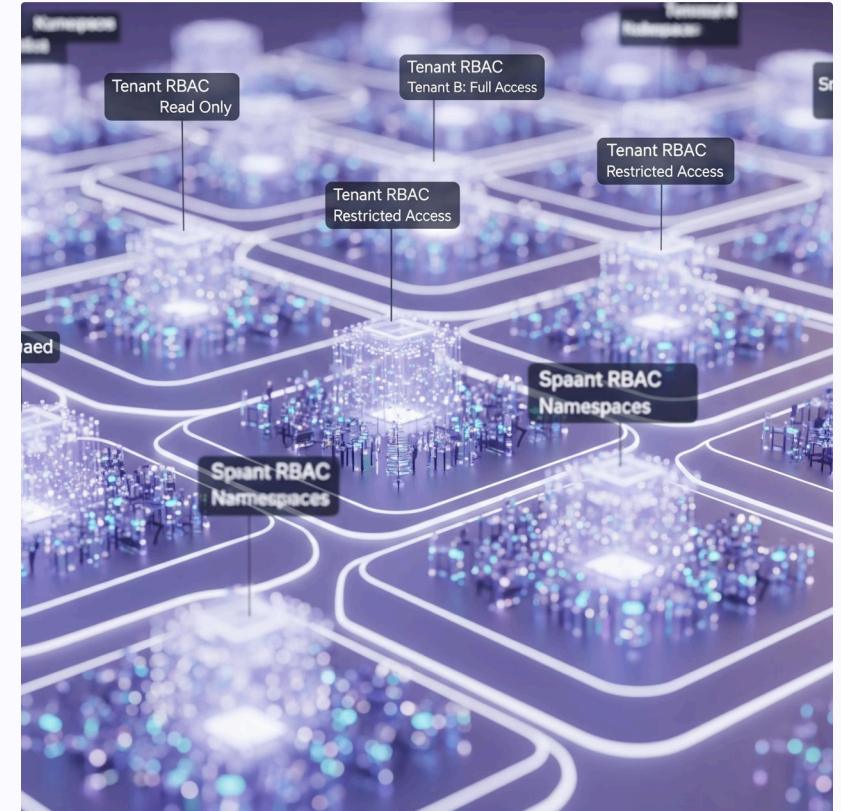
Kubernetes RBAC & Namespacing

Key Principles

- Separate **namespaces per team/tenant**
- Quotas & LimitRanges to prevent noisy-neighbor attacks
- RBAC principle: no cluster-admin; scope to namespace

RBAC (Read-Only to Pods)

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata: {name: viewer, namespace: tenant-a}
rules:
- apiGroups: [""]
  resources: ["pods","services"]
  verbs: ["get","list"]
---
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata: {name: bind-viewer, namespace: tenant-a}
subjects: [{kind: ServiceAccount, name: sa-app}]
roleRef: {kind: Role, name: viewer, apiGroup: rbac.authorization.k8s.io}
```



Secrets & Key Management

Secure Storage

Store secrets in **KMS/HSM**; inject via **External Secrets** operator

Rotation Policy

Regularly rotate credentials; prefer short-lived ephemeral tokens

Deployment Safety

Never bake secrets into container images or commit to repositories

External Secrets (K8s)

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: {name: infer-secrets}
spec:
  refreshInterval: 1h
  secretStoreRef: {name: aws-secrets, kind: ClusterSecretStore}
  target: {name: infer-env}
  data:
    - secretKey: API_KEY
  remoteRef: {key: prod/infer/api_key}
```

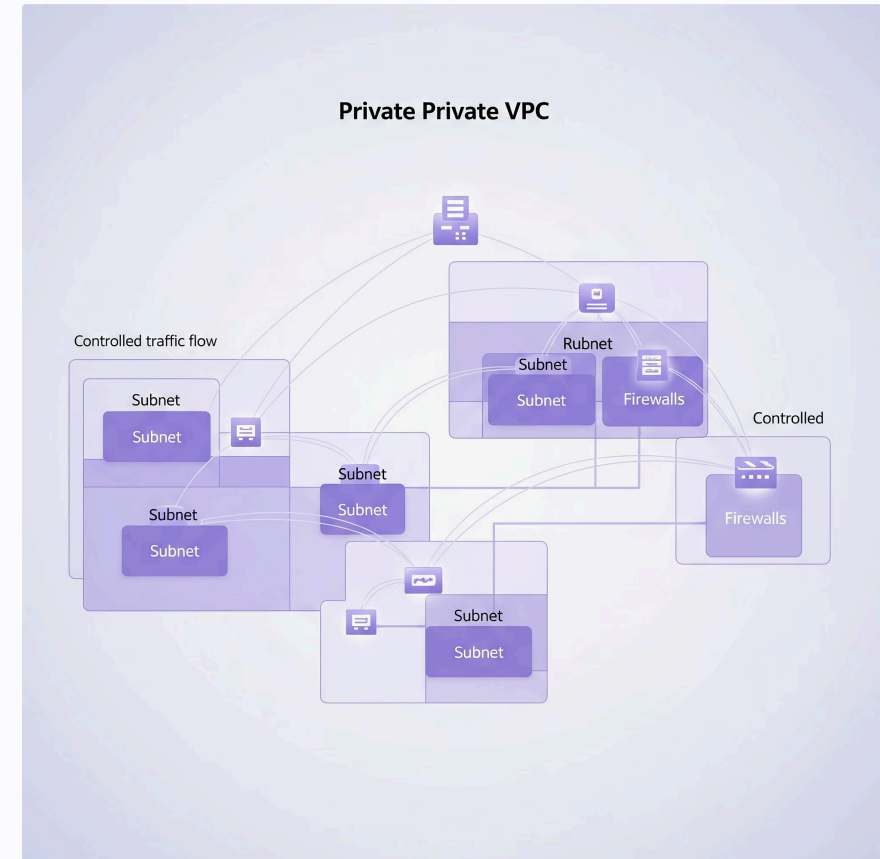
Network Security & Private Access

Network Controls

- **Private endpoints**; eliminate public IPs unless required
- Ingress allowlists; **NetworkPolicies** to limit east-west traffic
- Egress control: only allow S3/Blob/FeatureStore endpoints
- TLS everywhere with modern cipher suites only

K8s NetworkPolicy

```
kind: NetworkPolicy
apiVersion: networking.k8s.io/v1
metadata: {name: allow-only-gw, namespace: ml}
spec:
  podSelector: {matchLabels: {app: infer}}
  ingress:
    - from: [{podSelector: {matchLabels: {app: ingress-gw}}}]
  policyTypes: ["Ingress","Egress"]
```



Authorization Models

RBAC vs ABAC Approaches

RBAC (Role-Based)

- Roles map directly to resources/actions
- Simpler to implement and understand
- Examples: admin, reader, model-trainer

ABAC (Attribute-Based)

- Policies based on attributes (tenant, region, risk score)
- More flexible but complex to maintain
- Enables contextual security decisions

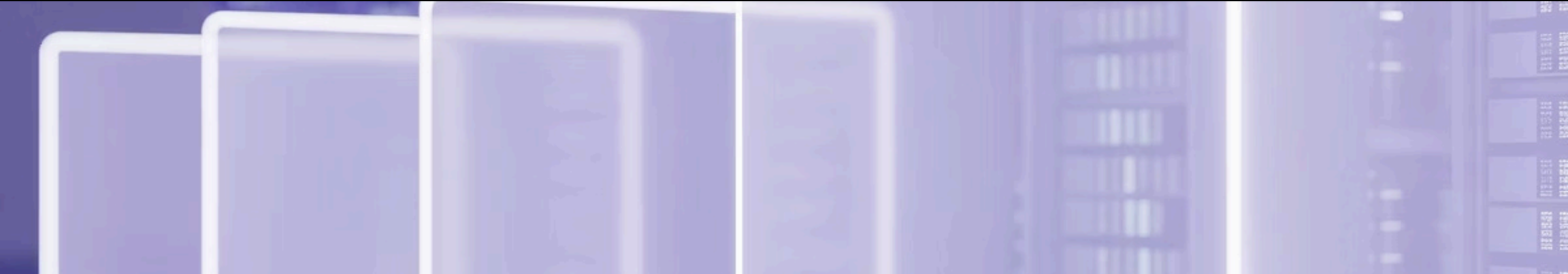
OPA (Rego) Policy Example

```
package authz

default allow = false

allow {
  input.claims.tenant == input.resource.tenant
  input.action == "infer"
  not input.resource.sensitive
}
```

Use **OPA/Rego** or cloud-native conditional policies to implement sophisticated authorization rules based on request context and resource attributes.



Multi-Tenant Isolation & Quotas

Tenant Boundaries

Dedicated namespace, service account, API key, and JWT scope per tenant

Usage Controls

Rate limits & quotas aligned with tenant SLAs and pricing tiers

Cost Attribution

Implement **cost tags** to track usage; automatically throttle abusive tenants

Model Separation

Deploy separate model weights per tenant when required by contract or regulations



Model-Aware Controls

Input Protection

- **Safe prompts** with policy enforcement
- **Content filters** as sidecars/middleware
- Input validation against injection attacks

Output & Usage Controls

- **Request budget:** cap tokens/images per minute
- Output filtering and sanitization
- **Watermarking/provenance** for generated content
- Cost monitoring and anomaly detection

❗ For LLMs, consider implementing a prompt/response chain that includes classifiers to detect and prevent both harmful inputs and outputs.



Audit, Forensics & Nonrepudiation

Comprehensive Logging

Record caller identity, scopes, resource ID, input hash, model version, latency, and response size

Immutable Storage

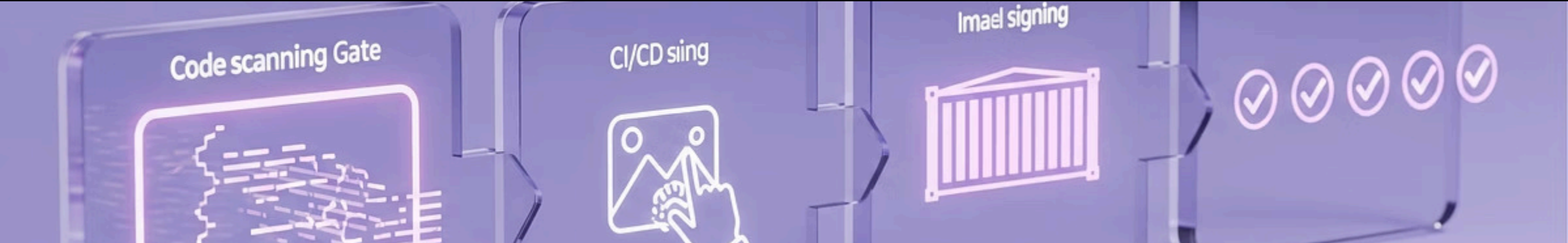
Use **WORM** (Write Once Read Many) or **S3 Object Lock** for tamper-proof audit trails

Distributed Tracing

Correlate **trace IDs** across gateway → service → datastore for end-to-end visibility

Anomaly Detection

Alert on suspicious patterns: QPS spikes, error bursts, scope misuse, unusual access patterns



Secure Delivery & Supply Chain

Supply Chain Security

- **Sign images/manifests** (cosign, Notation) + **SBOM** generation (Syft)
- Admission control to deny unsigned images, `:latest` tags, containers running as root
- Regular base image scanning and patching (CI gate)
- Dependency vulnerability scanning during build

Verification Example

```
# Verify signed container
cosign verify \
--key cosign.pub \
registry.example.com/models:v1.2.3

# Verify SBOM
cosign download sbom \
registry.example.com/models:v1.2.3
```



Red Team & Validation



Token Tests

Test token replay, expired token acceptance, missing audience checks



Network Tests

Attempt mTLS downgrade, rogue sidecar injection, cross-namespace access



Data Exfiltration

Try inter-tenant access; prompt injection attacks to extract sensitive data



API Fuzzing

Fuzz API payloads to verify schema validation and input sanitization

Establish a regular cadence of security validation exercises and incorporate findings into your security improvement roadmap.

Cost & Abuse Protections

Financial Controls

- Per-key **daily caps** & **spike arrest** thresholds
- Dynamic pricing tiers mapped to **rate limits** + **model choices**
- Budget alerts and automatic cutoffs

Abuse Detection

- Detect grey noise: high QPS, low utility → throttle/ban
- ML-based anomaly detection for usage patterns
- Shadow billing dashboard for tenants to monitor their own usage
- Graduated response: warn → throttle → suspend

⚠ A single compromised API key without rate limiting can lead to tens of thousands of dollars in unexpected charges within hours.



Quick Start Checklist

Capstone Implementation

1 Edge Security

Private endpoint + WAF + rate limiting

2 Identity

OAuth2/OIDC for users; mTLS for services

3 Authorization

JWT scopes: model:infer, model:admin

4 Kubernetes

RBAC + NetworkPolicies per namespace

5 Secrets

External Secrets + KMS; no static keys in code

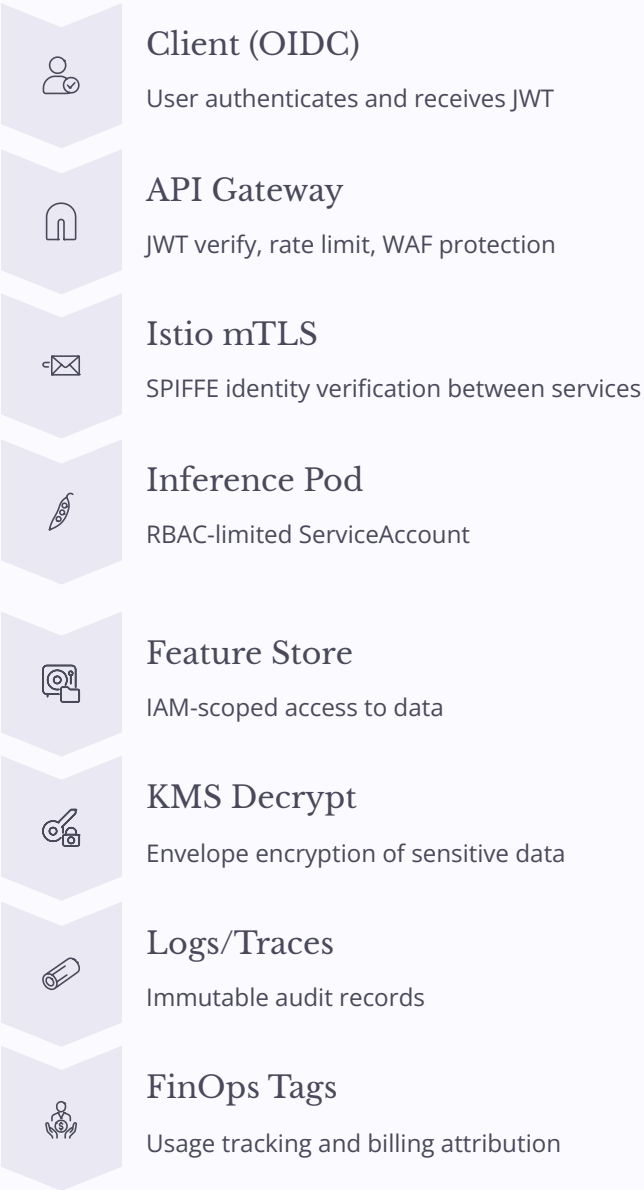
6 Observability

Audit logs + trace IDs + anomaly alerts

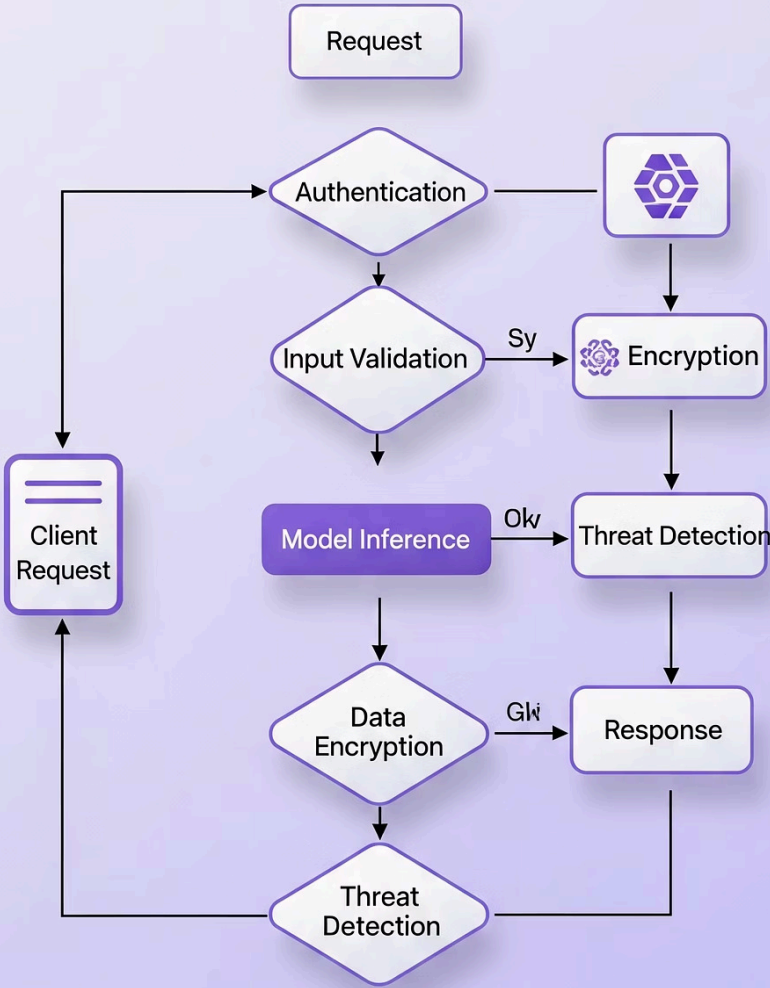
7 Deployment

Canary policy rollout; signed images only

Example "Golden" Request Flow



Security Controls





Discussion Prompt

"For your Capstone, which identity path will you implement first: OAuth2/OIDC at the edge or mTLS + SPIFFE inside the mesh? Why?"

Consider factors like implementation complexity, security coverage, ecosystem support, and your organization's expertise.